

SYSTEM DESIGN CACHING & CDN

DOSEN PENGAMPU:

RUNAL REZKIAWAN, S.KOM.,M.T



Disusun Oleh:

FAUZIAH

105841104223

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan " **SYSTEM DESIGN CACHING & CDN** " sebagai salah satu tugas pada mata kuliah **Scalable Systems Design**.

Laporan ini membahas proses implementasi sistem caching menggunakan bahasa pemrograman Go dengan memanfaatkan Redis sebagai media cache dan MySQL sebagai database utama. Selain itu, laporan ini juga menjelaskan langkah-langkah instalasi, konfigurasi, implementasi program, serta hasil pengujian beberapa strategi caching, yaitu Cache Aside, Read Through, Write Through, dan Refresh Ahead.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca.

Makassar, 26 Juni 2026

Penulis

A. Materi Ringkas

Perkembangan teknologi digital menuntut sistem mampu memberikan akses data yang cepat dan stabil. Banyaknya permintaan data secara berulang sering menyebabkan beban tinggi pada server dan database sehingga performa sistem menurun. Untuk mengatasi masalah tersebut digunakan teknik caching, yaitu penyimpanan sementara data yang sering diakses agar proses pengambilan data menjadi lebih cepat dan efisien. Teknologi caching seperti Redis dan Memcached banyak digunakan untuk meningkatkan performa sistem dan mengurangi beban server.

Caching bekerja dengan menyimpan sementara data yang sering diakses sehingga aplikasi tidak perlu selalu mengambil data dari database, sedangkan CDN atau content delivery network merupakan jaringan server yang tersebar di berbagai lokasi untuk mendistribusikan konten seperti, video, CSS, dan javascript dari server yang paling dekat dengan pengguna. Dengan menggabungkan caching dan CDN, waktu akses menjadi lebih cepat, beban server utama berkurang, penggunaan bandwidth lebih efisien, dan pengalaman pengguna menjadi lebih baik.

Dalam pengembangan aplikasi ini digunakan tiga teknologi utama yang memiliki peran penting :

1. GO (golang)

Go atau Golang adalah bahasa pemrograman yang dikembangkan oleh Google untuk membangun aplikasi yang cepat, sederhana, dan efisien. Go memiliki performa yang tinggi karena merupakan bahasa yang dikompilasi (compiled language) serta mendukung pemrosesan banyak tugas secara bersamaan (concurrency). Dalam proyek ini, Go digunakan sebagai backend yang bertugas menerima permintaan dari pengguna, menghubungkan aplikasi dengan Redis dan MySQL, mengolah data, kemudian mengirimkan hasilnya kembali kepada pengguna melalui API.

2. MySQL

MySQL adalah sistem manajemen basis data relasional (RDBMS) yang digunakan untuk menyimpan data secara permanen. Semua data utama,

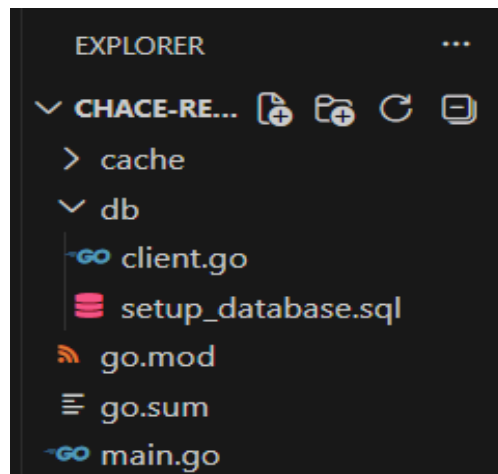
seperti data pengguna, produk, atau informasi lainnya, disimpan di dalam MySQL sebagai sumber data utama (source of truth).

3. Redis

Redis adalah sistem penyimpanan data berbasis memori (in-memory database) yang umumnya digunakan sebagai cache. Karena data disimpan di RAM, proses pembacaan data menjadi jauh lebih cepat dibandingkan mengambil data langsung dari database utama. Pada program ini, Redis berfungsi menyimpan data yang sering diakses sehingga ketika ada permintaan yang sama, aplikasi dapat mengambil data langsung dari Redis tanpa harus mengakses MySQL.

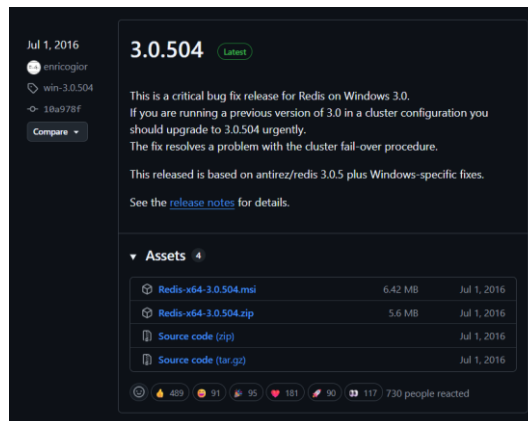
B. Langkah-langkah Pemrograman

1. Struktur file

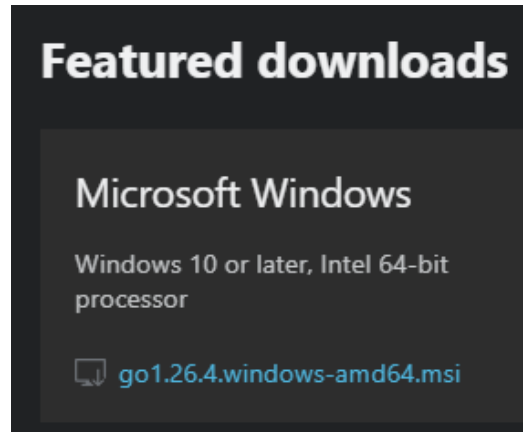


2. Instalasi Software

- Redis



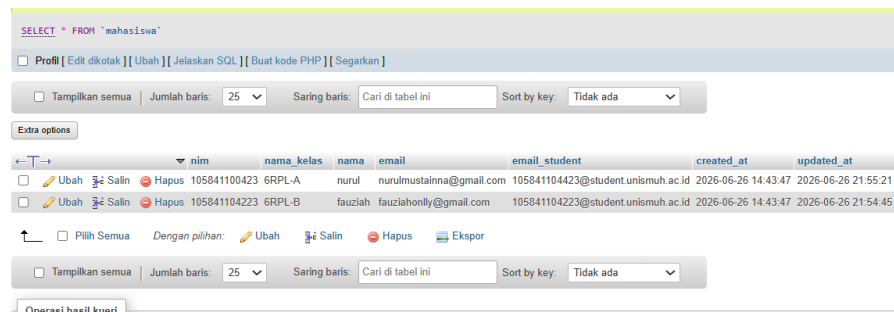
- Go



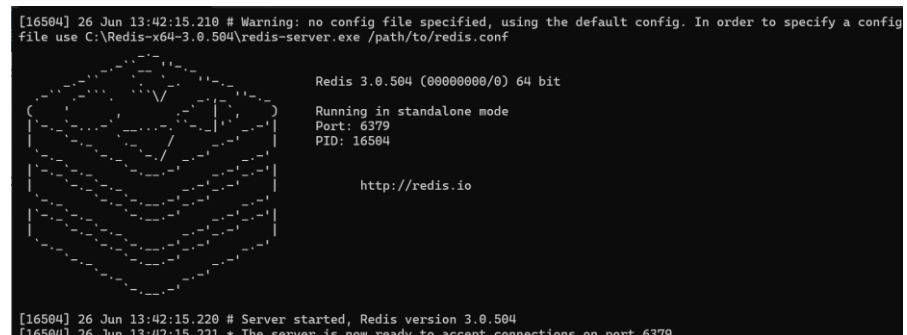
- MySQL



3. Membuat Database MySQL dengan nama chacheredis



4. Menjalankan Redis Server



5. Membuat koneksi di main.go

```
func main() {
    ctx := context.Background()

    mysqlDSN := "root:@tcp(localhost:3306)/chacheredis?parseTime=true&charset=utf8mb4"
    database, err := db.NewDB(mysqlDSN)
    if err != nil {
        fmt.Printf("Error koneksi MySQL: %v\n", err)
        return
    }
    defer database.Close()
}
```

6. Membuat strategi caching

```
redisCacheAside := cache.NewRedisCache(redisAddr)
ca := cache.NewCacheAside(redisCacheAside, database, 60*time.Second)

redisCacheReadThrough := cache.NewRedisCache(redisAddr)
rt := cache.NewReadThrough(redisCacheReadThrough, database, 60*time.Second)

redisCacheWriteThrough := cache.NewRedisCache(redisAddr)
wt := cache.NewWriteThrough(redisCacheWriteThrough, database, 60*time.Second)

redisCacheWriteBack := cache.NewRedisCache(redisAddr)
wb := cache.NewWriteBack(redisCacheWriteBack, database, 60*time.Second, 100)
defer wb.Close()
```

7. Membuat REST API

```
C:\chace-redis-learn-main>go run .
[SERVER] Berjalan di port :8080 dengan REDIS CONTAINER dan MySQL Database.
=== Available Endpoints ===
1. Cache-Aside:      GET/POST /api/cache-aside
2. Read-Through:    GET /api/read-through
3. Write-Through:    GET/POST /api/write-through
4. Write-Back:       GET/POST /api/write-back
5. Write-Back Flush: POST /api/write-back/flush
6. Refresh-Ahead:    GET /api/refresh-ahead
7. Large Data:       GET /api/large-data
8. Cache Eviction:   POST /api/cache/evict?policy=lru|lfu|fifo
9. Cache Validation: GET /api/cache/validate?nim=xxx
10. Reset Cache:     GET /api/reset
=====
```

C. Source Code

Source code dari main.go :

```

1 func main() {
2     ctx := context.Background()
3
4     mysqlDBM := "root:@tcp(localhost:3306)/chacheredis?parseTime=true&charset=utf8mb4"
5     database, err := db.NewDB(mysqlDBM)
6     if err != nil {
7         fmt.Printf("error koneksi mysql: %v\n", err)
8         return
9     }
10    defer database.Close()
11
12    fallbackData := []db.Mahasiswa{
13        {Nim: "0801104422", Nama: "nurul", Email: "nurulnista2@gmail.com", EmailStudent: "109801104422@student.unismuh.ac.id"},
14        {Nim: "0801104423", Nama: "fauziah", Email: "fauziahonliye@gmail.com", EmailStudent: "109801104423@student.unismuh.ac.id"},
15    }
16    for _, s := range fallbackData {
17        _ = database.Save(ctx, s)
18    }
19
20    defaultNim := "109801104421"
21    redisAddr := "localhost:6379"
22
23    redisCacheAside := cache.NewRedisCache(redisAddr)
24    ca := cache.NewCacheAside(redisCacheAside, database, 60*time.Second)
25
26    redisCacheReadThrough := cache.NewRedisCache(redisAddr)
27    rt := cache.NewReadThrough(redisCacheReadThrough, database, 60*time.Second)
28
29    redisCacheWriteThrough := cache.NewRedisCache(redisAddr)
30    wt := cache.NewWriteThrough(redisCacheWriteThrough, database, 60*time.Second)
31
32    redisCacheWriteBack := cache.NewRedisCache(redisAddr)
33    wb := cache.NewWriteBack(redisCacheWriteBack, database, 60*time.Second, 100)
34    defer wb.Close()
35
36    redisCacheRefreshAhead := cache.NewRedisCache(redisAddr)
37    ra := cache.NewRefreshAhead(redisCacheRefreshAhead, database, 20*time.Second, 0.5)
38
39    http.HandleFunc("/api/cache-aside", func(w http.ResponseWriter, r *http.Request) {
40        w.Header().Set("Content-Type", "application/json")
41
42        if r.Method == http.MethodGet {
43            nim := r.URL.Query().Get("nim")
44            if nim == "" {
45                nim = defaultNim
46            }
47            start := time.Now()
48            m, err := ca.GetMahasiswa(ctx, nim)
49            dur := time.Since(start)
50
51            if err != nil {
52                w.WriteHeader(http.StatusNotFound)
53                json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
54                return
55            }
56
57            hits, misses := redisCacheAside.GetStats()
58            validationStats := redisCacheAside.GetValidationStats()
59            json.NewEncoder(w).Encode(Response{
60                Data: m,
61                Latency: dur.String(),
62                Stats: Stats{
63                    Strategy: "Cache Aside (Redis)",
64                    DBHits: database.GetQueryCount(),
65                    CacheHits: hits,
66                    CacheMisses: misses,
67                    ValidationStats: &validationStats,
68                },
69            })
70        } else if r.Method == http.MethodPost {
71            var m db.Mahasiswa
72            if err := json.NewDecoder(r.Body).Decode(&m); err != nil {
73                w.WriteHeader(http.StatusBadRequest)
74                json.NewEncoder(w).Encode(map[string]string{"error": "Invalid JSON"})
75                return
76            }
77
78            start := time.Now()
79            err := ca.SaveMahasiswa(ctx, m)
80            dur := time.Since(start)
81
82            if err != nil {
83                w.WriteHeader(http.StatusInternalServerError)
84                json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
85                return
86            }
87
88            json.NewEncoder(w).Encode(map[string]string{
89                "message": "Data saved successfully (Cache-Aside: DB updated, cache invalidated)",
90                "latency": dur.String(),
91                "data": m,
92            })
93        }
94    })
95 }

```

```

1 // 2. ENDPOINT: READ-THROUGH (GET untuk Baca)
2 http.HandleFunc("/api/read-through", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4     if r.Method == http.MethodGet {
5         nim := r.URL.Query().Get("nim")
6         if nim == "" {
7             nim = defaultNim
8         }
9         start := time.Now()
10        m, err := rt.GetMahasiswa(ctx, nim)
11        dur := time.Since(start)
12
13        if err != nil {
14            w.WriteHeader(http.StatusNotFound)
15            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
16            return
17        }
18        hits, misses := redisCacheReadThrough.GetStats()
19        validationStats := redisCacheReadThrough.GetValidationStats()
20        json.NewEncoder(w).Encode(Response{
21            Data: m,
22            Latency: dur.String(),
23            Stats: Stats{
24                Strategy: "Read-Through (Redis)",
25                DBHits: database.GetQueryCount(),
26                CacheHits: hits,
27                CacheMisses: misses,
28                ValidationStats: &validationStats,
29            },
30        })
31    }
32 })

```

```

1 // 3. ENDPOINT: WRITE-THROUGH (GET untuk Baca, POST untuk Update)
2 http.HandleFunc("/api/write-through", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4
5     if r.Method == http.MethodGet {
6         nim := r.URL.Query().Get("nim")
7         if nim == "" {
8             nim = defaultNim
9         }
10        start := time.Now()
11        m, err := wt.GetMahasiswa(ctx, nim)
12        dur := time.Since(start)
13
14        if err != nil {
15            w.WriteHeader(http.StatusNotFound)
16            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
17            return
18        }
19
20        hits, misses := redisCacheWriteThrough.GetStats()
21        validationStats := redisCacheWriteThrough.GetValidationStats()
22        json.NewEncoder(w).Encode(Response{
23            Data: m,
24            Latency: dur.String(),
25            Stats: Stats{
26                Strategy: "Write-Through (Redis)",
27                DBHits: database.GetQueryCount(),
28                CacheHits: hits,
29                CacheMisses: misses,
30                ValidationStats: &validationStats,
31            },
32        })
33    } else if r.Method == http.MethodPost {
34        var m db.Mahasiswa
35        if err := json.NewDecoder(r.Body).Decode(&m); err != nil {
36            w.WriteHeader(http.StatusBadRequest)
37            json.NewEncoder(w).Encode(map[string]string{"error": "Invalid JSON"})
38            return
39        }
40
41        start := time.Now()
42        err := wt.SaveMahasiswa(ctx, m)
43        dur := time.Since(start)
44
45        if err != nil {
46            w.WriteHeader(http.StatusInternalServerError)
47            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
48            return
49        }
50
51        json.NewEncoder(w).Encode(map[string]interface{}{
52            "message": "Data saved successfully (Write-Through: DB and cache updated synchronously)",
53            "latency": dur.String(),
54            "data": m,
55        })
56    }
57 })

```

```

1 // 4. ENDPOINT: WRITE-BACK (GET untuk Baca, POST untuk Update)
2 http.HandleFunc("/api/write-back", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4
5     if r.Method == http.MethodGet {
6         nim := r.URL.Query().Get("nim")
7         if nim == "" {
8             nim = defaultNim
9         }
10        start := time.Now()
11        m, err := wb.GetMahasiswa(ctx, nim)
12        dur := time.Since(start)
13
14        if err != nil {
15            w.WriteHeader(http.StatusNotFound)
16            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
17            return
18        }
19
20        hits, misses := redisCacheWriteBack.GetStats()
21        validationStats := redisCacheWriteBack.GetValidationStats()
22        queueLen := wb.GetQueueLength()
23        json.NewEncoder(w).Encode(Response{
24            Data: m,
25            Latency: dur.String(),
26            Stats: Stats{
27                Strategy: "Write-Back (Redis)",
28                DBHits: database.GetQueryCount(),
29                CacheHits: hits,
30                CacheMisses: misses,
31                ValidationStats: &validationStats,
32                QueueLength: &queueLen,
33            },
34        })
35    } else if r.Method == http.MethodPost {
36        var m db.Mahasiswa
37        if err := json.NewDecoder(r.Body).Decode(&m); err != nil {
38            w.WriteHeader(http.StatusBadRequest)
39            json.NewEncoder(w).Encode(map[string]string{"error": "Invalid JSON"})
40            return
41        }
42
43        start := time.Now()
44        err := wb.SaveMahasiswa(ctx, m)
45        dur := time.Since(start)
46
47        if err != nil {
48            w.WriteHeader(http.StatusInternalServerError)
49            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
50            return
51        }
52
53        queueLen := wb.GetQueueLength()
54        json.NewEncoder(w).Encode(map[string]interface{}{
55            "message": "Data saved successfully (Write-Back: Cache updated immediately, DB queued)",
56            "latency": dur.String(),
57            "data": m,
58            "queue_length": queueLen,
59        })
60    }
61 })
62

```

```

1 // 5. ENDPOINT: WRITE-BACK FLUSH (POST untuk memaksa flush queue)
2 http.HandleFunc("/api/write-back/flush", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4     if r.Method == http.MethodPost {
5         start := time.Now()
6         err := wb.Flush(ctx)
7         dur := time.Since(start)
8
9         if err != nil {
10            w.WriteHeader(http.StatusInternalServerError)
11            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
12            return
13        }
14
15        json.NewEncoder(w).Encode(map[string]interface{}{
16            "message": "Write-back queue flushed successfully",
17            "latency": dur.String(),
18        })
19    }
20 })

```

```

1 // 6. ENDPOINT: REFRESH-AHEAD (GET untuk Baca)
2 http.HandleFunc("/api/refresh-ahead", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4     if r.Method == http.MethodGet {
5         nim := r.URL.Query().Get("nim")
6         if nim == "" {
7             nim = defaultNim
8         }
9         start := time.Now()
10        m, err := ra.GetMahasiswa(ctx, nim)
11        dur := time.Since(start)
12
13        if err != nil {
14            w.WriteHeader(http.StatusNotFound)
15            json.NewEncoder(w).Encode(map[string]string{"error": err.Error()})
16            return
17        }
18
19        // Mengembalikan info TTL tambahan
20        var remainingStr string = "not found/expired"
21        val, found := redisCacheRefreshAhead.Get(ctx, nim)
22        if found {
23            var item cache.RefreshItem
24            if err := cache.UnmarshalValue(val, &item); err == nil {
25                elapsed := time.Since(item.CachedAt)
26                remaining := item.TTL - elapsed
27                remainingStr = remaining.String()
28            }
29        }
30
31        hits, misses := redisCacheRefreshAhead.GetStats()
32        validationStats := redisCacheRefreshAhead.GetValidationStats()
33        json.NewEncoder(w).Encode(map[string]interface{}{
34            "data":      m,
35            "latency":    dur.String(),
36            "remaining_ttl": remainingStr,
37            "stats": Stats{
38                Strategy:    "Refresh-Ahead (Redis)",
39                DBHits:        database.GetQueryCount(),
40                CacheHits:      hits,
41                CacheMisses:    misses,
42                ValidationStats: &validationStats,
43            },
44        })
45    }
46 })

```

```

1 http.HandleFunc("/api/large-data", func(w http.ResponseWriter, r *http.Request) {
2     w.Header().Set("Content-Type", "application/json")
3     if r.Method == http.MethodGet {
4         start := time.Now()
5
6         // 1. Cek apakah seluruh data besar tersimpan di Redis
7         val, found := redisCacheReadThrough.Get(ctx, "large_students_data")
8         if found {
9             var cachedStudents []db.Mahasiswa
10            if err := cache.UnmarshalValue(val, &cachedStudents); err == nil {
11                dur := time.Since(start)
12                hits, misses := redisCacheReadThrough.GetStats()
13                json.NewEncoder(w).Encode(map[string]interface{}{
14                    "message": "Data besar berhasil diambil dari REDIS CONTAINER (Cache Hit)",
15                    "latency": dur.String(),
16                    "count":   len(cachedStudents),
17                    "stats": Stats{
18                        Strategy:    "Redis-Large-Data",
19                        DBHits:        database.GetQueryCount(),
20                        CacheHits:      hits,
21                        CacheMisses:    misses,
22                    },
23                    "data": cachedStudents,
24                })
25                return
26            }
27        }
28
29        time.Sleep(500 * time.Millisecond)
30        freshStudents := database.GetAllLocal()
31
32        redisCacheReadThrough.Set(ctx, "large_students_data", freshStudents, 120*time.Second)
33
34        dur := time.Since(start)
35        hits, misses := redisCacheReadThrough.GetStats()
36        json.NewEncoder(w).Encode(map[string]interface{}{
37            "message": "Data besar diambil dari DATABASE (Cache Miss)",
38            "latency": dur.String(),
39            "count":   len(freshStudents),
40            "stats": Stats{
41                Strategy:    "Redis-Large-Data",
42                DBHits:        database.GetQueryCount(),
43                CacheHits:      hits,
44                CacheMisses:    misses,
45            },
46            "data": freshStudents,
47        })
48    }
49 })
50

```

```

1 // 8. ENDPOINT: CACHE EVICTION (POST untuk trigger eviction)
2 http.HandleFunc("/api/cache/evict", func(w http.ResponseWriter, r *http.Request) {
3     w.Header().Set("Content-Type", "application/json")
4     if r.Method == http.MethodPost {
5         policy := r.URL.Query().Get("policy")
6         var evictionPolicy cache.EvictionPolicy
7
8         switch policy {
9             case "lru":
10                 evictionPolicy = cache.LRU
11             case "lfu":
12                 evictionPolicy = cache.LFU
13             case "fifo":
14                 evictionPolicy = cache.FIFO
15             default:
16                 evictionPolicy = cache.LRU
17         }
18
19         start := time.Now()
20         evictedAside := redisCacheAside.Evict(evictionPolicy)
21         evictedRT := redisCacheReadThrough.Evict(evictionPolicy)
22         evictedWT := redisCacheWriteThrough.Evict(evictionPolicy)
23         evictedWB := redisCacheWriteBack.Evict(evictionPolicy)
24         evictedRA := redisCacheRefreshAhead.Evict(evictionPolicy)
25         dur := time.Since(start)
26
27         totalEvicted := evictedAside + evictedRT + evictedWT + evictedWB + evictedRA
28
29         json.NewEncoder(w).Encode(map[string]interface{}{
30             "message": fmt.Sprintf("Cache eviction completed using %s policy", policy),
31             "latency": dur.String(),
32             "evicted": map[string]int{
33                 "cache_aside": evictedAside,
34                 "read_through": evictedRT,
35                 "write_through": evictedWT,
36                 "write_back": evictedWB,
37                 "refresh_ahead": evictedRA,
38                 "total": totalEvicted,
39             },
40         })
41     }
42 })
43

```

```

1 http.HandleFunc("/api/cache/validate", func(w http.ResponseWriter, r *http.Request) {
2     w.Header().Set("Content-Type", "application/json")
3     if r.Method == http.MethodGet {
4         nim := r.URL.Query().Get("nim")
5         if nim == "" {
6             w.WriteHeader(http.StatusBadRequest)
7             json.NewEncoder(w).Encode(map[string]string{"error": "nim parameter is required"})
8             return
9         }
10
11         validAside := redisCacheAside.Validate(ctx, nim)
12         validRT := redisCacheReadThrough.Validate(ctx, nim)
13         validWT := redisCacheWriteThrough.Validate(ctx, nim)
14         validWB := redisCacheWriteBack.Validate(ctx, nim)
15         validRA := redisCacheRefreshAhead.Validate(ctx, nim)
16
17         json.NewEncoder(w).Encode(map[string]interface{}{
18             "nim": nim,
19             "validation_results": map[string]bool{
20                 "cache_aside": validAside,
21                 "read_through": validRT,
22                 "write_through": validWT,
23                 "write_back": validWB,
24                 "refresh_ahead": validRA,
25             },
26             "validation_stats": map[string]cache.ValidationStats{
27                 "cache_aside": redisCacheAside.GetValidationStats(),
28                 "read_through": redisCacheReadThrough.GetValidationStats(),
29                 "write_through": redisCacheWriteThrough.GetValidationStats(),
30                 "write_back": redisCacheWriteBack.GetValidationStats(),
31                 "refresh_ahead": redisCacheRefreshAhead.GetValidationStats(),
32             },
33         })
34     }
35 })
36

```

```

1 http.HandleFunc("/api/reset", func(w http.ResponseWriter, r *http.Request) {
2     w.Header().Set("Content-Type", "application/json")
3     database.ResetCounter()
4     redisCacheAside.Clear()
5     redisCacheReadThrough.Clear()
6     redisCacheWriteThrough.Clear()
7     redisCacheWriteBack.Clear()
8     redisCacheRefreshAhead.Clear()
9
10    json.NewEncoder(w).Encode(map[string]string{
11        "message": "Semua data cache di Redis Container dibersihkan & hit counter di-reset",
12    })
13 })
14
15 // Start server
16 port := ":8080"
17 fmt.Printf("[SERVER] Berjalan di port %s dengan REDIS CONTAINER dan MySQL Database.\n", port)
18 fmt.Println("=== Available Endpoints ===")
19 fmt.Println("1. Cache-Aside:      GET/POST /api/cache-aside")
20 fmt.Println("2. Read-Through:      GET /api/read-through")
21 fmt.Println("3. Write-Through:      GET/POST /api/write-through")
22 fmt.Println("4. Write-Back:         GET/POST /api/write-back")
23 fmt.Println("5. Write-Back Flush:   POST /api/write-back/flush")
24 fmt.Println("6. Refresh-Ahead:     GET /api/refresh-ahead")
25 fmt.Println("7. Large Data:         GET /api/large-data")
26 fmt.Println("8. Cache Eviction:     POST /api/cache/evict?policy=lru|lfu|fifo")
27 fmt.Println("9. Cache Validation:   GET /api/cache/validate?nim=xxx")
28 fmt.Println("10. Reset Cache:       GET /api/reset")
29 fmt.Println("=====")
30
31 if err := http.ListenAndServe(port, nil); err != nil {
32     fmt.Printf("Gagal memulai server: %v\n", err)
33 }
34 }
35

```

D. Output

Setelah seluruh proses instalasi, konfigurasi, dan implementasi selesai dilakukan, tahap berikutnya adalah menjalankan aplikasi serta melakukan pengujian terhadap setiap komponen yang digunakan. Pengujian dilakukan untuk memastikan bahwa Redis Server, MySQL Database, dan aplikasi Go telah berjalan dengan baik serta setiap strategi caching dapat berfungsi sesuai dengan yang diharapkan. Berikut merupakan hasil output dari proses pengujian tersebut. Redis Server dijalankan terlebih dahulu untuk memastikan layanan cache aktif dan siap menerima koneksi dari aplikasi Go. Setelah Redis berhasil dijalankan, akan muncul tampilan log seperti berikut.

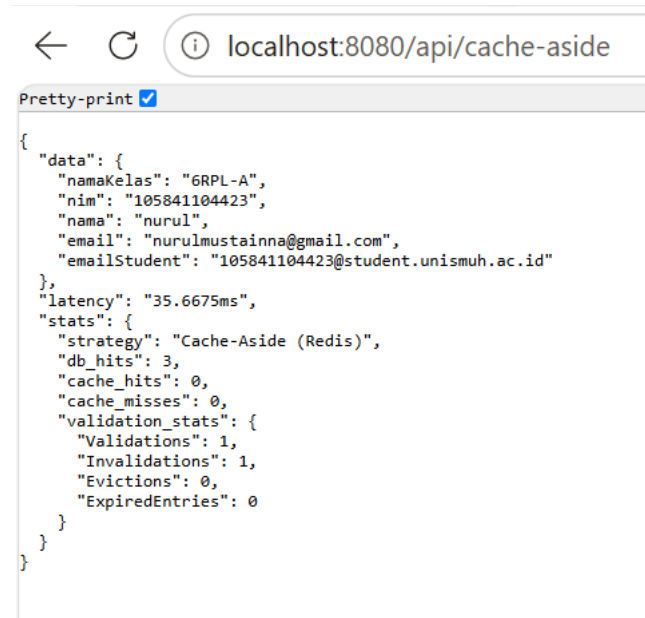
```

[16504] 26 Jun 13:42:15.220 # Server started, Redis version 3.0.504
[16504] 26 Jun 13:42:15.221 * The server is now ready to accept connections on port 6379
[16504] 26 Jun 14:46:49.818 * 1 changes in 3600 seconds. Saving...
[16504] 26 Jun 14:46:49.984 * Background saving started by pid 15236
[16504] 26 Jun 14:46:50.196 # fork operation complete
[16504] 26 Jun 14:46:50.208 * Background saving terminated with success
[16504] 26 Jun 15:46:51.029 * 1 changes in 3600 seconds. Saving...
[16504] 26 Jun 15:46:51.542 * Background saving started by pid 26372
[16504] 26 Jun 15:46:51.873 # fork operation complete
[16504] 26 Jun 15:46:51.874 * Background saving terminated with success
[16504] 26 Jun 16:46:52.082 * 1 changes in 3600 seconds. Saving...
[16504] 26 Jun 16:46:52.167 * Background saving started by pid 17248
[16504] 26 Jun 16:46:52.392 # fork operation complete
[16504] 26 Jun 16:46:52.413 * Background saving terminated with success
[16504] 26 Jun 22:28:31.842 * 1 changes in 3600 seconds. Saving...
[16504] 26 Jun 22:28:31.878 * Background saving started by pid 10028
[16504] 26 Jun 22:28:32.091 # fork operation complete
[16504] 26 Jun 22:28:32.092 * Background saving terminated with success

```

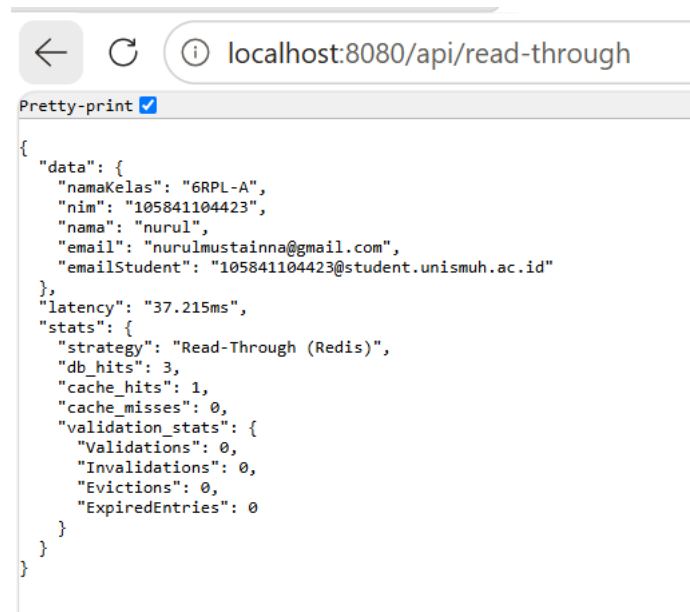
Selanjutnya pengujian terhadap masing-masing endpoint menggunakan browser atau Postman. Hasil pengujian menunjukkan bahwa setiap endpoint berhasil diakses dan menghasilkan respons sesuai dengan strategi caching yang diimplementasikan. Sebagai berikut

1. Cache-aside



```
{
  "data": {
    "namaKelas": "6RPL-A",
    "nim": "105841104423",
    "nama": "nurul",
    "email": "nurulmustainna@gmail.com",
    "emailStudent": "105841104423@student.unismuh.ac.id"
  },
  "latency": "35.6675ms",
  "stats": {
    "strategy": "Cache-Aside (Redis)",
    "db_hits": 3,
    "cache_hits": 0,
    "cache_misses": 0,
    "validation_stats": {
      "Validations": 1,
      "Invalidations": 1,
      "Evictions": 0,
      "ExpiredEntries": 0
    }
  }
}
```

2. Read-through



```
{
  "data": {
    "namaKelas": "6RPL-A",
    "nim": "105841104423",
    "nama": "nurul",
    "email": "nurulmustainna@gmail.com",
    "emailStudent": "105841104423@student.unismuh.ac.id"
  },
  "latency": "37.215ms",
  "stats": {
    "strategy": "Read-Through (Redis)",
    "db_hits": 3,
    "cache_hits": 1,
    "cache_misses": 0,
    "validation_stats": {
      "Validations": 0,
      "Invalidations": 0,
      "Evictions": 0,
      "ExpiredEntries": 0
    }
  }
}
```

3. Write-through

```
localhost:8080/api/write-through

Pretty-print ☒

{
  "data": {
    "namaKelas": "6RPL-A",
    "nim": "105841104423",
    "nama": "nurul",
    "email": "nurulmustainna@gmail.com",
    "emailStudent": "105841104423@student.unismuh.ac.id"
  },
  "latency": "34.58ms",
  "stats": {
    "strategy": "Write-Through (Redis)",
    "db_hits": 5,
    "cache_hits": 0,
    "cache_misses": 0,
    "validation_stats": {
      "Validations": 1,
      "Invalidations": 1,
      "Evictions": 0,
      "ExpiredEntries": 0
    }
  }
}
```

4. Refresh-ahead

```
localhost:8080/api/refresh-ahead

Pretty-print ☒

{
  "data": {
    "namaKelas": "",
    "nim": "",
    "nama": "",
    "email": "",
    "emailStudent": ""
  },
  "latency": "36.5725ms",
  "remaining_ttl": "-2562047h47m16.854775807s",
  "stats": {
    "strategy": "Refresh-Ahead (Redis)",
    "db_hits": 4,
    "cache_hits": 2,
    "cache_misses": 0,
    "validation_stats": {
      "Validations": 0,
      "Invalidations": 0,
      "Evictions": 0,
      "ExpiredEntries": 0
    }
  }
}
```

E. Kesimpulan

Berdasarkan hasil implementasi dan pengujian, dapat disimpulkan bahwa aplikasi caching menggunakan Go, Redis, dan MySQL berhasil dijalankan dengan baik. Go berperan sebagai backend yang menyediakan REST API, MySQL digunakan sebagai penyimpanan data utama (database), sedangkan Redis berfungsi sebagai media cache untuk menyimpan data yang sering diakses sehingga proses pengambilan data menjadi lebih cepat.

Implementasi beberapa strategi caching, yaitu Cache Aside, Read Through, Write Through, dan Refresh Ahead, menunjukkan bahwa setiap strategi memiliki mekanisme kerja yang berbeda dalam mengelola data antara Redis dan MySQL. Cache Aside mengambil data dari Redis terlebih dahulu sebelum mengakses MySQL, Read Through memungkinkan cache mengambil data dari database secara otomatis ketika cache kosong, Write Through menjaga konsistensi data dengan menulis ke Redis dan MySQL secara bersamaan, sedangkan Refresh Ahead memperbarui cache sebelum masa berlaku data habis agar data tetap tersedia dengan cepat.

Hasil pengujian menunjukkan bahwa Redis mampu mengurangi akses langsung ke MySQL sehingga waktu respons aplikasi menjadi lebih cepat, terutama ketika terjadi cache hit. Selain itu, penggunaan Redis juga membantu mengurangi beban database karena data yang sering diakses tidak selalu diambil dari MySQL. Dengan demikian, penerapan strategi caching menggunakan Redis dapat meningkatkan performa, efisiensi, dan skalabilitas aplikasi, khususnya pada sistem yang memiliki frekuensi akses data yang tinggi.